Istanbul Technical University

EHB326E – Assignment 2 Report

Dijkstra Algorithm and microBee Profile Selection

Arda Yüce (040230045)

3 November 2026

1 Abstract

This study implements Dijkstra's single-source shortest-path algorithm in RV32IMC assembly on the configurable microBee processor and compares execution performance against FPGA resource cost. A seven-node functional test and a deterministic 32-node graph were verified in Vivado XSim against independent reference algorithms. CPU cycle/retirement counters and branch events provided execution measurements, while Vivado synthesis reported resource utilization for the Arty A7-100 target. Five configurations passed verification and synthesis; a planned gshare configuration was excluded after an XSim elaboration incompatibility was traced to protected core RTL. A weighted-sum multi-criteria evaluation selected the reduced-DMEM profile (P6) for the tested workload: it retained the reference execution time while lowering BRAM use by 25%. This selection is conditional on the application's reduced 32 KiB data-memory footprint.

2 Objective and Requirements

The task is to execute Dijkstra's algorithm correctly on microBee and choose a processor configuration that balances computation cost with limited FPGA resources. Dijkstra's algorithm finds minimum-cost routes from a source in a non-negative weighted graph [1]. For a fixed workload, the study aims to (i) implement and verify the algorithm in assembly, (ii) compare the effect of configurable branch prediction and data-memory size, and (iii) select a suitable profile by an explicit multi-criteria decision procedure. All candidate designs retain the provided processor ISA, pipeline and core RTL, in accordance with course rules [3]. Identical software, benchmark inputs, measurement boundaries and FPGA target are used for quantitative comparison.

3 Method and System Design

3.1 Algorithm and SoC organization

A square adjacency matrix represents the graph; a linear scan chooses the unvisited node with minimum tentative distance, followed by relaxation of its outgoing edges. The algorithm maintains `dist`, `visited` and `parent` arrays. Its time complexity is $O(N^2)$ with $O(N^2)$ matrix storage. This variant was chosen instead of a priority queue because its addressing, control flow and memory layout are simple to implement and inspect in assembly. Absent edges use `0xFFFFFFFF`; unreachable distances use `0x7FFFFFFF`. The implementation also guards against distance overflow.

The processor executes instructions from IMEM at `0x80000000` and reads the graph from DMEM at `0x20000000`. Completion and status words, plus the result arrays, are checked by an external testbench. Figure 1 summarizes the actual SoC organization, including the AXI UART Lite, clock/reset path and read-only simulation observation. The 100 MHz clock is the configured simulation/clocking target, *not* a demonstrated post-route timing result. microBee architectural and configuration details follow the official project sources [2].

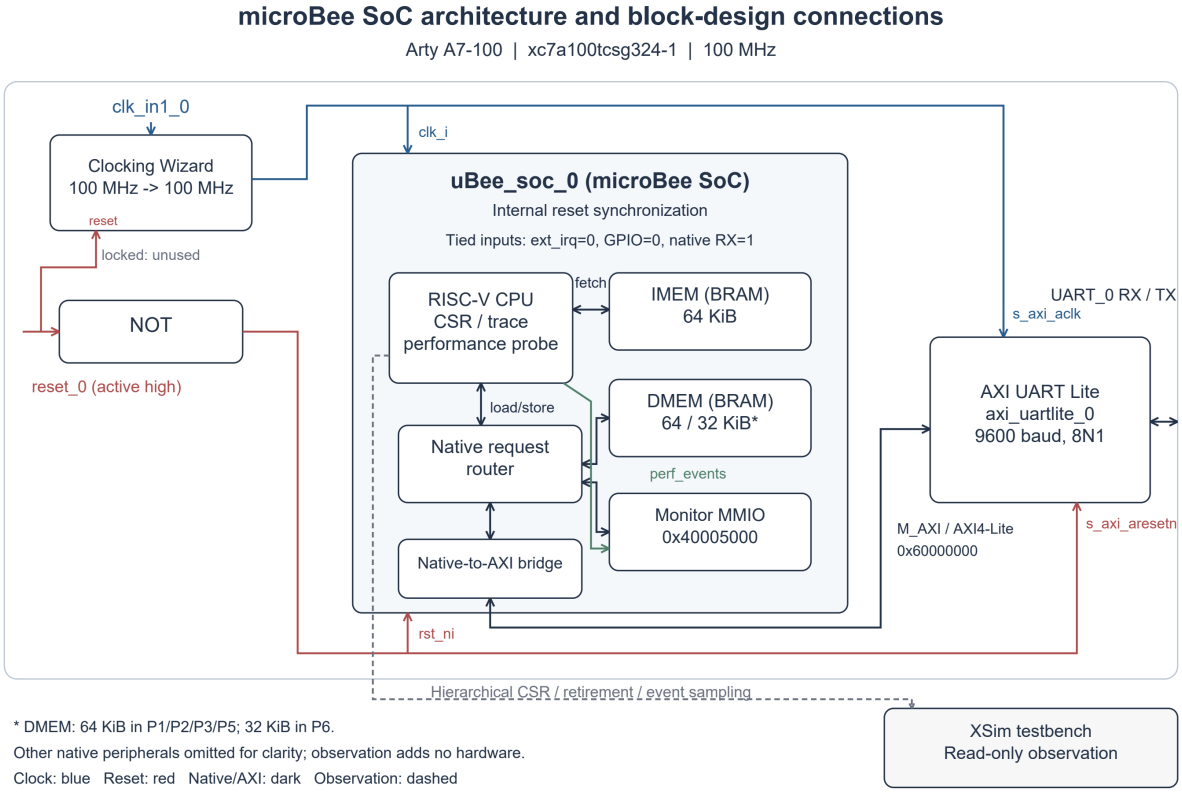


Figure 1: microBee SoC, memory/peripheral paths and non-intrusive RTL observation. The diagram summarizes the tested block design, not physical-board validation.

3.2 Implementation and configuration choices

A single RV32IMC-compatible assembly image reads graph size, source and matrix, initializes the working arrays, then repeatedly selects and relaxes vertices. The completed distances and predecessors are stored in DMEM, followed by `done/status` publication. The tested software was kept unchanged between processor profiles. Table 1 specifies the controlled parameters. P1 is the baseline; P2/P3 alter predictor capacity, P4 was intended to evaluate gshare, P5 disables prediction, and P6 halves DMEM capacity. All profiles retain `IMEM_WORDS=16384`, `MONITOR_ENABLE=1`, `MUL_ENABLE=1` and `DIV_ENABLE=1`; other parameters remain at their defaults.

Table 1: Planned processor profiles (P4 not quantitatively evaluated).

Profile	BP enable	Entries	Gshare	GHR width	DMEM words
P1	1	128	0	5	16384
P2	1	32	0	5	16384
P3	1	256	0	5	16384
P4 (planned)	1	128	1	5	16384
P5	0	128	0	5	16384
P6	1	128	0	5	8192

4 Simulation and Findings

4.1 Benchmarks and functional behavior

A seven-node graph initially tested path reconstruction and an unreachable vertex. From source 0, the expected distance vector was $[0,7,9,20,20,11,\infty]$; the shortest route to node 4 was $0\to 2\to 5\to 4$ with cost 20. Both the initial run and the reset/reload run passed.

The common performance workload was a deterministic, undirected 32-node graph constructed as a 4×8 grid with selected diagonal edges (Figure 2). Its 70 edges represent 14.11% of all $32\cdot 31/2 = 496$ possible undirected pairs. Edge weights are non-negative and nonuniform, source node 0 is fixed, and the graph/memory-image data are identical across profiles. This controls differences unrelated to the CPU configuration.

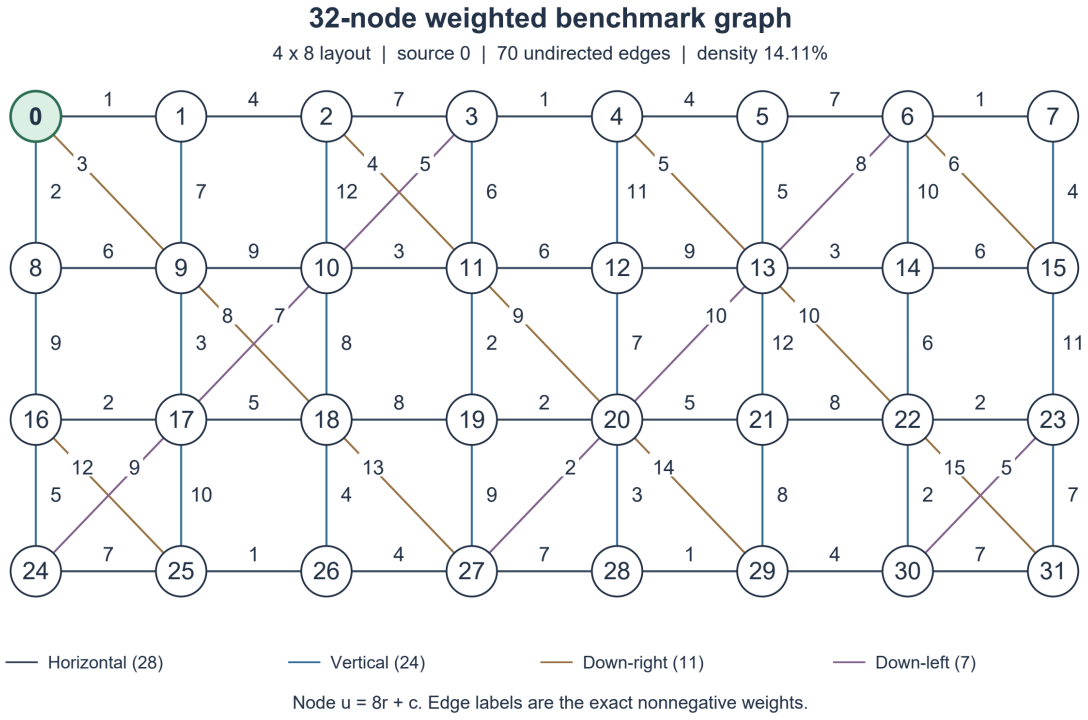


Figure 2: Deterministic 32-node benchmark: actual topology and weights, with source node 0 highlighted.

4.2 Performance measurement

Vivado/XSim 2025.2 was used to observe the real RTL execution. The measurement interval extends from the accepted fetch of the first algorithm instruction to retirement of the final exit branch. It excludes reset, memory-image loading, result publication and the terminal idle loop. The 64-bit `mcycle` and `minstret` differences give window cycles and retired instructions; branch/taken/correct/misprediction events are collected from existing RTL signals without inserting measurement instructions into the assembly program. Cycle counts were cross-checked with clock edges and timestamps; retirements and branch events were checked against the execution trace. The testbench accessed the existing hardware CSR counter signals through hierarchical RTL references, including the mcycle and minstret debug outputs. Therefore, performance measurement was performed externally during simulation rather than through CSR-reading instructions embedded in the assembly program.

The 32-node interval includes 17,531 retired instructions, of which 17,528 belong to Dijkstra; three preceding preparation instructions retire inside the window because the pipeline overlaps the measurement boundary. Thus $\mathrm{CPI}_{\mathrm{window}} = \Delta mcycle/\Delta minstret$ is a *window-based* value, not an isolated drained-kernel CPI. All five measured profiles retire 5,324 conditional branches (3,961 taken and 1,363 not taken). Table 2 and Figure 3 report the observed values and whole-SoC post-synthesis utilization.

Table 2: 32-node performance and Arty A7-100 synthesis results. BRAM is in RAMB36-equivalent tiles.

Profile	Cycles	CPI	Miss (%)	LUT	FF	BRAM	DSP
P1	42,044	2.398	16.28	6,569	4,482	32	4
P2	42,139	2.404	16.89	6,299	4,284	32	4
P3	42,044	2.398	16.28	6,947	4,739	32	4
P5	48,528	2.768	N/A	6,019	4,023	32	4
P6	42,044	2.398	16.28	6,574	4,473	24	4

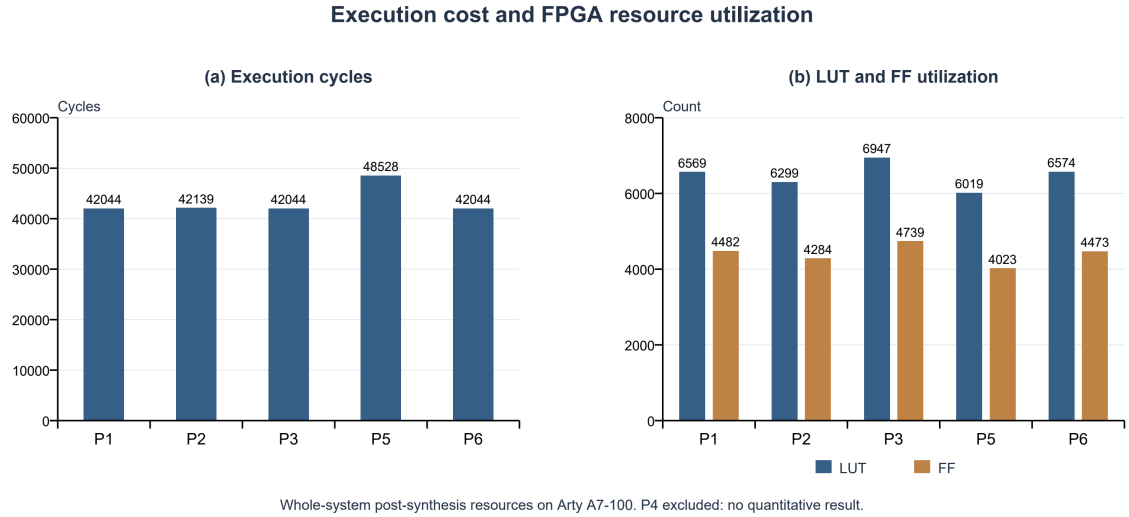


Figure 3: Execution cycles and LUT/FF counts for the five successfully evaluated profiles.

5 Verification and Evaluation

5.1 Correctness, interpretation and limitations

XSim results for P1, P2, P3, P5 and P6 passed distance, predecessor-chain, visitation, completion/status, trap and timeout checks. Independent Bellman–Ford and Floyd–Warshall calculations confirmed expected shortest paths; two reset/reload passes agreed for each verified profile. The testbench also checked instruction activity so that correct outputs could not be attributed merely to preloaded data.

P2 uses 4.1% fewer LUTs than P1 but incurs 32 additional branch mispredictions (899 vs. 867) and 95 additional cycles (+0.23%). Their ratio is approximately 2.97 cycles per additional misprediction, not an independently measured penalty. P3 provides no prediction or cycle benefit despite higher resource use. Disabling prediction (P5) increases execution time by 15.4% while reducing logic resources; prediction accuracy is not applicable to P5. P6 retains P1's 42,044 cycles while reducing BRAM from 32 to 24 tiles with nearly unchanged LUT/FF usage. CPI also includes other execution stalls, which were not separately characterized.

P4 was intended to test gshare prediction. Its original RTL activated an XSim 2025.2 elaboration error because the simulator does not support directly slicing a concatenation expression. The failure was traced to the global-history update and a functionally equivalent assignment/truncation form was identified as a possible compatibility fix. However, applying it would require editing protected microBee core RTL; course rules forbid such modification [3]. Consequently, P4 was not evaluated or assigned fabricated results. The proposed fix was *not* used in the reported experiments.

P6 halves configured DMEM from 64 to 32 KiB. The tested program makes no stack accesses and its actual data accesses fit within the smaller region, so the observed result is valid for this benchmark. Nevertheless, BRAM savings are a *direct consequence of provisioning less memory*, not a stand-alone architectural efficiency improvement; the result cannot be generalized to programs requiring more than 32 KiB. Utilization figures are post-synthesis whole-SoC results, not timing-closed implementation or physical FPGA operation.

5.2 Multi-criteria profile selection

Four lower-is-better criteria were selected: execution cycles (weight 0.40), LUTs (0.30), flip-flops (0.15), and BRAM tiles (0.15). Resource use therefore receives 60% aggregate importance, reflecting FPGA limitations, while execution speed remains the most important single criterion. CPI and prediction errors were analyzed separately rather than counted again as correlated performance criteria; DSP was constant across profiles and omitted.

For a profile i and cost criterion j, the weighted-sum model (WSM) uses

$$r_{ij} = \frac{\min_k x_{kj}}{x_{ij}}, \qquad S_i = \sum_{j=1}^{4} w_j r_{ij}, \qquad \sum_j w_j = 1.$$

Higher S_i is preferred. Table 3 gives the resulting ranking, based only on experimentally valid profiles.

P6 ranks first because it maintains reference execution time while reducing required BRAM for this particular benchmark. P2 is the runner-up and may be attractive where 64 KiB DMEM capacity must be retained. WSM weights express the design priorities rather than being optimized to force a particular winner; the ranking is specific to the stated criteria and workload. As a robustness check, the profile ranking was also evaluated using TOPSIS and alternative

Table 3: WSM ranking under the declared 40/30/15/15 weights.

Rank	Profile	Score (0–1)	Score (0–100)
1	P6	0.9596	95.96
2	P2	0.9391	93.91
3	P1	0.9220	92.20
4	P5	0.9091	90.91
5	P3	0.8998	89.98

criterion weights. TOPSIS ranked P6 first (0.7125), followed by P2 (0.6159). P6 also remained the preferred profile under four tested weight scenarios. In a 20,000-sample random-weight analysis, P6 ranked first in approximately 68% of cases, followed by P5 (23%) and P2 (9%). Thus, P6 is robust across the tested scenarios, although the selection is not entirely independent of criterion weights.

5.3 Reproducibility

The assembly image, seven-node and 32-node data, independent reference checks and testbenches were prepared for delivery using the course's `Homework_N` structure. The same executable and graph were used for all valid profiles, and archived simulation/synthesis records were checked for consistency. Supplementary scripts, configuration files, and archived evidence are documented through the companion project repository: github.com/candela2411/EHB326E-HW2-040230045. The packaged materials passed host-level build/data/hash checks; a *new clean Vivado project replay from the assembled package was not performed.*

6 Conclusion

The Dijkstra assembly program passed functional and repeatability checks on five microBee profiles. A fixed benchmark and aligned RTL measurement interval revealed competing costs. Disabling prediction saved logic resources but increased runtime; enlarging the predictor did not improve this workload. The WSM analysis chose P6, which cut BRAM use by 25% through a 50% DMEM-capacity reduction without changing the observed 42,044-cycle execution time. This is a valid application-specific configuration decision, not a guarantee for larger-memory programs. No placement/routing, timing closure, bitstream or physical FPGA test is claimed.

References

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. https://doi.org/10.1007/BF01386390.

[2] GSTL-ITU, *uBee: Open-Source RISC-V Soft-Core Processor*, project source and documentation, 2026. https://github.com/GSTL-ITU/uBee.

[3] Istanbul Technical University, EHB326E, *General Rules*, Fall 2026–2027 course handout.